Python CCEE Cheatsheet: Sessions 8–10

Advanced OOP, Exception Handling & Logging

SESSION 8 & 9: Advanced Python & OOP

1. Classes and Objects Fundamentals

Class: Blueprint/template for creating objects
Object: Concrete instance of a class

class Student:
def **init**(self, name, marks): # constructor
self.name = name # instance attribute
self.marks = marks

```
def show(self):                    # method
    print(self.name, self.marks)
```

s1 = Student("Rahul", 90) # create object
s1.show() # call method

Key Points:

- `__init__`: constructor, called when object is created
- `self`: reference to the current object instance
- Attributes: data stored in objects
- Methods: functions defined in classes

---

2. Instance vs Class Variables ⚠ MCQ Trap

Instance variables: Unique to each object, defined in `__init__` via `self`
Class variables: Shared by all instances, defined directly in class body

class A:
x = 10 # CLASS variable

a1 = A()
a2 = A()
a1.x = 20 # creates INSTANCE variable on a1
print(A.x, a2.x) # Output: 10 10 (not 20 20!)

Trap: `a1.x = 20` doesn't change `A.x`; it creates `a1.x` (instance), shadowing the class variable.

---

3. Object References

Everything in Python is an object; variables are just **labels/references** to objects.

x = 10
y = x # x and y reference SAME int object (10)
x = 20 # x now references a NEW object
print(y) # Still 10

For mutable objects (lists, dicts):
lst1 = [1, 2, 3]
lst2 = lst1 # both reference SAME list
lst2.append(4)
print(lst1) # [1, 2, 3, 4] - modified!

---

4. Mutable Default Arguments ⚠ CRITICAL MCQ Trap

Problem: Default mutable argument (list, dict) is created **once** at function definition, shared by all calls.

def test(x=[]):
x.append(1)
return x

print(test()) # [1]
print(test()) # [1, 1] - TRAP! Same list reused
print(test()) # [1, 1, 1]

Solution: Use `None` as default:
def test(x=None):
if x is None:
x = []
x.append(1)
return x

print(test()) # [1]
print(test()) # [1] - Fresh list each time

This is a **classic CCEE trap**; always watch for mutable defaults!

---

5. Data Hiding (Encapsulation)

Single underscore `_attr`: Convention, means "use at your own risk"
Double underscore `__attr`: Name-mangled to `_ClassName__attr`, harder to access

class Account:
def **init**(self, bal):
self.__balance = bal # "hidden"

```
def deposit(self, amt):
    self.__balance += amt
```

```
def get_balance(self):
    return self.__balance
```

a = Account(1000)
a.deposit(500)
print(a.get_balance()) # 1500
print(a.__balance) # AttributeError ✓

But this WORKS (name-mangling):

print(a._Account__balance) # 1500 (breaks encapsulation!)

Key: `__` is name-mangling, not true privacy; it's a convention to enforce encapsulation.

---

6. Inheritance & Overriding

Inheritance: Child class reuses/extends parent class

class Animal:
def speak(self):
print("Some sound")

class Dog(Animal):
def speak(self): # overriding
print("Bark")

d = Dog()
d.speak() # Bark

Overriding: Same method name/signature in child, different implementation

Overloading in Python: True overloading by signature **does NOT exist**; last definition wins.

class Test:
def add(self, a, b):
return a + b

```
def add(self, a, b, c):       # overwrites previous add()
    return a + b + c
```

t = Test()
t.add(1, 2) # TypeError! add() expects 3 args (self + 2)

Workaround: Use default/variable arguments:
def add(self, a, b, c=0):
return a + b + c

t.add(1, 2) # 3
t.add(1, 2, 3) # 6

---

7. Operator Overloading (Special Methods)

Redefine operators like `+`, `-`, `==` using special methods (`__add__`, `__sub__`, `__eq__`):

class Vector:
def **init**(self, x, y):
self.x, self.y = x, y

```
def __add__(self, other):
    return Vector(self.x + other.x, self.y + other.y)

def __str__(self):
    return f"({self.x}, {self.y})"
```

v1 = Vector(1, 2)
v2 = Vector(3, 4)
v3 = v1 + v2 # calls **add**
print(v3) # (4, 6)

---

8. Regular Expressions (regex)

Import `re` module; common functions:

Function	Purpose
`re.match(pattern, string)`	Match at **START** of string only
`re.search(pattern, string)`	First match **ANYWHERE**
`re.findall(pattern, string)`	All non-overlapping matches
`re.compile(pattern)`	Return compiled regex object (for reuse)

Common patterns:

- `\d` = digit, `\w` = word char, `\s` = whitespace
- `+` = 1+ occurrences, `*` = 0+ occurrences
- `()` = groups (capture substrings)

import re

match vs search

m1 = re.match(r"\d+", "abc123") # None (digits not at START)
m2 = re.search(r"\d+", "abc123") # Match "123" (found ANYWHERE)

groups

m = re.search(r"(\w+)@(\w+).com", "mail@cdac.com")
print(m.group(0)) # mail@cdac.com (full match)
print(m.group(1), m.group(2)) # mail cdac (subgroups)

findall

results = re.findall(r"\d+", "a1b2c3")
print(results) # ['1', '2', '3']

Trap: `match()` checks only at the **start**; use `search()` if you need any position.

---

9. Generators

Generator: Function using `yield` to produce values lazily, one at a time.

def countdown(n):
while n > 0:
yield n
n -= 1

for x in countdown(3):
print(x) # 3, 2, 1

Key differences:

- `return`: ends function, returns single value
- yield: pauses function, remembers state, resumes on next iteration
- Calling generator function returns **generator object**, not the first value
- Generator can be iterated **only once** (unless recreated)

def gen():
yield 1
yield 2
return 3 # becomes StopIteration(3)

g = gen()
for x in g:
print(x) # 1, 2 (return value NOT printed in loop)

next() works too

g2 = gen()
print(next(g2)) # 1
print(next(g2)) # 2
print(next(g2)) # StopIteration exception

Memory efficient: Generates values on-the-fly, not all at once like lists.

---

10. Decorators

Decorator: Function that takes another function and returns a **modified** version, often using an inner wrapper.

def logger(func):
def wrapper(*args, **kwargs):
print("Calling", func.**name**)
result = func(*args, **kwargs)
print("Done")
return result # ⚠ MUST return!
return wrapper

@logger # syntactic sugar for: add = logger(add)
def add(a, b):
return a + b

add(2, 3)

Output:

Calling add

Done

Trap: If `wrapper` forgets `return result`, the decorated function returns `None` even if original returned something.

def bad_deco(func):
def wrapper(*a, **k):
print("start")
func(*a, **k) # no return!
print("end")
return wrapper

@bad_deco
def hello():
return "hi"

x = hello()
print(x) # None (not "hi"!)

Common uses: Logging, timing, authentication, validation.

SESSION 10: Exception Handling & Logging

1. Basic Try-Except

try:
x = int(input("Enter number: "))
y = 10 / x
except ZeroDivisionError:
print("Cannot divide by zero")
except ValueError:
print("Invalid integer")

If exception in `try` matches an `except` clause, that clause runs; program continues after.

Catch multiple types:
except (TypeError, ValueError):
print("Type or Value Error")

Catch any exception (not recommended):
except Exception:
print("Any exception")

---

2. Try-Except-Else-Finally

try:
risky_code()
except SomeError:
handle_error()
else:
print("No exception occurred")
finally:
cleanup() # ALWAYS runs

- `else`: Runs only if **NO exception** in `try`
- `finally`: Runs **ALWAYS**, even if exception or `return` in `try`/`except`

Trap: If both `try` and `finally` have `return`, `finally`'s `return` overrides.

def func():
try:
return 1
finally:
return 2 # overrides return 1

print(func()) # 2

---

3. Try-Finally (Resource Cleanup)

Use when you must clean up (close files, release locks) regardless of exceptions:

```
f = open("data.txt")
try:
data = f.read()
except FileNotFoundError:
print("File not found")
finally:
f.close() # ALWAYS runs
```

Even if exception occurs in `try`, `finally` is guaranteed to run before propagating.

Modern approach (context manager):

```
with open("data.txt") as f:
data = f.read() # automatically closes f
```

---

4. Raising Exceptions

Raise built-in exception:

```
if age < 0:
raise ValueError("Age cannot be negative")
```

Re-raise caught exception:

```
try:
risky_operation()
except ZeroDivisionError as e:
print("Caught:", e)
raise # re-raise same exception
```

---

5. User-Defined Exceptions

Custom exceptions inherit from `Exception`:

```
class MyError(Exception):
pass

class InvalidAgeError(Exception):
def init(self, message):
self.message = message

def check_age(age):
if age < 0:
raise InvalidAgeError("Age must be non-negative")

try:
check_age(-1)
except InvalidAgeError as e:
print("Custom error:", e.message)
```

---

6. Logging vs Print

Print: Simple output, no levels, no filtering, no files.
Logging: Severity levels, filtering, file/console routing, timestamps.

```
import logging

logging.basicConfig(
level=logging.INFO,
format="%(asctime)s - %(levelname)s - %(message)s"
)

logging.debug("Debug info") # NOT shown (level is INFO)
logging.info("Process started") # SHOWN
logging.warning("Low disk") # SHOWN
logging.error("Failed") # SHOWN
logging.critical("System down") # SHOWN
```

Severity levels (ascending):

1. `DEBUG` (10)
2. `INFO` (20)
3. `WARNING` (30) — default level
4. `ERROR` (40)
5. `CRITICAL` (50)

Trap: `logging.debug()` won't show by default unless you set `level=logging.DEBUG`.

Quick Reference: Common Traps & Tricks for CCEE MCQs

Topic	Trap	Example
Class vs Instance vars	`a1.x = 20` creates instance var, doesn't change class var	`A.x == 10`, `a1.x == 20`, `a2.x == 10`
Mutable defaults	Shared across all function calls	`def f(x=[]): x.append(1)` → `[1]`, `[1,1]`, ...
re.match vs re.search	`match` only at START; `search` ANYWHERE	`re.match(r"\d", "a1")` → None
Generator return	return in generator becomes StopIteration, not printed in loop	`for x in gen():` doesn't print the return value
Decorator return	Wrapper must `return result` or decorated func returns None	Forgetting `return` breaks function
finally overrides	finally return overrides try return	try: return 1; finally: return 2 → 2
Name mangling	`__attr` → `_ClassName__attr` (convention, not absolute)	`a._Account__balance` still works!
except bare	Bare except: catches **everything**, including KeyboardInterrupt	Use specific exception types
Unhandled exceptions	If no except matches, exception propagates up; program may crash	`except TypeError:` won't catch `ValueError`
Logging level	Only messages >= set level are shown	level=INFO → debug NOT shown

---

Code Snippet Recap: Session 8–10

Classes & OOP:
class Parent:
x = 10
def **init**(self, val):
self.y = val

```
def method(self):
return self.y

class Child(Parent):
def method(self):
return self.y * 2

c = Child(5)
print(c.method(), Child.x) # 10 10
```

Generators:

```
def gen():
for i in range(3):
yield i ** 2

g = gen()
print(list(g)) # [0, 1, 4]
print(list(g)) # [] (exhausted)
```

Decorators:

```
def timer(func):
def wrapper(*a, **k):
import time
start = time.time()
result = func(*a, **k)
print(f"Time: {time.time() - start}")
return result
return wrapper

@timer
def slow_add(a, b):
return a + b
```

Regex:

```
import re
text = "Email: john@example.com"
m = re.search(r"(\w+)@(\w+.\w+)", text)
print(m.group(1), m.group(2)) # john example.com
```

Exception Handling:

```
try:
result = 10 / int(input("Enter divisor: "))
except ValueError:
print("Enter a valid number")
except ZeroDivisionError:
print("Cannot divide by zero")
else:
print("Result:", result)
finally:
print("Done")
```

Logging:

```
import logging
```

```
logging.basicConfig(level=logging.WARNING)
logging.warning("This shows") # Shows
logging.info("This doesn't") # Doesn't show
```

Key Definitions for Quick Revision

- **Object:** Instance of a class with specific attributes
- **Class:** Blueprint/template for objects
- **Inheritance:** Child class reuses parent class features
- **Overriding:** Redefining parent method in child
- **Encapsulation:** Hiding internal details; use `__attr` for convention
- **Generator:** Lazy iterator using `yield`
- **Decorator:** Function that modifies/wraps another function
- **Exception:** Runtime error event; handled with try-except
- **Finally:** Block that runs regardless of exception
- **Logging:** Structured way to record app events by severity

---

Last-Minute Tips for CCEE Exams

1. **Watch class vs instance variables:** Most common trick.
2. **Watch mutable defaults:** Shared across calls.
3. **`regex` match vs search:** Know the difference.
4. **Decorator must return:** Or function returns None.
5. **Finally always runs:** Even with return or exception.
6. **Generators iterate once:** Unless recreated.
7. **Except specific types:** Avoid bare `except:`.
8. **User exceptions inherit `Exception`:** For standard handling.
9. **Logging levels matter:** DEBUG won't show at INFO level.
10. **Name mangling is convention:** `__attr` is not absolute privacy.

Good luck with CCEE! Review these traps, code snippets, and definitions before the exam. Practice with similar code-based MCQs.